

SHRI GURU TEGH BAHADUR INSTITUTE OF
MANAGEMENT AND INFORMATION
TECHNOLOGY



MINOR PROJECT

On

**STUDENT RESULT
MANAGEMENT SYSTEM
(PYTHON)**

Submitted To :- Pabhjot Kaur Ma'am

Submitted By :- Shaurya Baijel

BCA 5B

(05590202019)

ABSTRACT

Student Result Management System is a project which aims in developing a computerized system to maintain all the records of student, their courses and results. This project has many features which are generally not available in normal student management system like facility of student n teacher login, student n teacher register, manage n view students, manage n view courses, manage n view results, logout, exit etc.. It also have facility to update the total number of courses and student and results on dashboard. It has also a facility where student after logging in their accounts can see list of student registered and their respective courses. Moreover it also have facility of search through which we can search a student by their roll number and courses by initial letters of courses.

Overall, this project of ours is being developed to help the students as well as staff to view and maintain the records of students and and their courses and results.

INTRODUCTION

This system saves the time of the student and of the administrator. It includes processes like registration of the student's details, assigning the department based on their course, and maintenance of the record.

This makes the system easy to handle and feasible for finding the omission with updating at the same time. As for the existing system, they use to maintain their record manually which makes it vulnerable to security. If filed a query to search or update in a manual system, it will take a lot of time to process the query and make a report which is a tedious job.

As the system used in the institute is outdated as it requires paper, files, and binders, which will require the human workforce to maintain them. To get registered in the institute, a student in this system should come to the university. Get the forms from the counter while standing in the queue which consumes a lot of the student's time as well as of the management team.

As the number of the student increases in the institute manually managing the strength becomes a hectic job for the administrator. This computerized system stores all the data in the database which makes it easy to fetch and update whenever needed.

SYSTEM OBJECTIVES

Improvement in control and performance :

The system is developed to cope up with the current issues and problems of managing Student Records. The system can add, delete, update Student Records and Courses Records

Save cost :

After computerized system is implemented less human force will be required to maintain the Student Records, thus reducing the overall cost.

Save time:

User is able to search student record by using few clicks of mouse and few search keywords thus saving their valuable time.

SOFTWARE REQUIREMENTS

Operating system:

Windows 7 is used as the operating system as it is stable and supports more features and is more user friendly

Database SQLite3:

SQLite3 is used as database as it easy to maintain and retrieve records by simple queries which are in English language which are easy to understand and easy to write.

Development tools and Programming language:

PYTHON is used to write the whole code and develop frames and designs on tkinter module for GUI. The IDE used is Pycharm for python language.

HARDWARE REQUIREMENTS

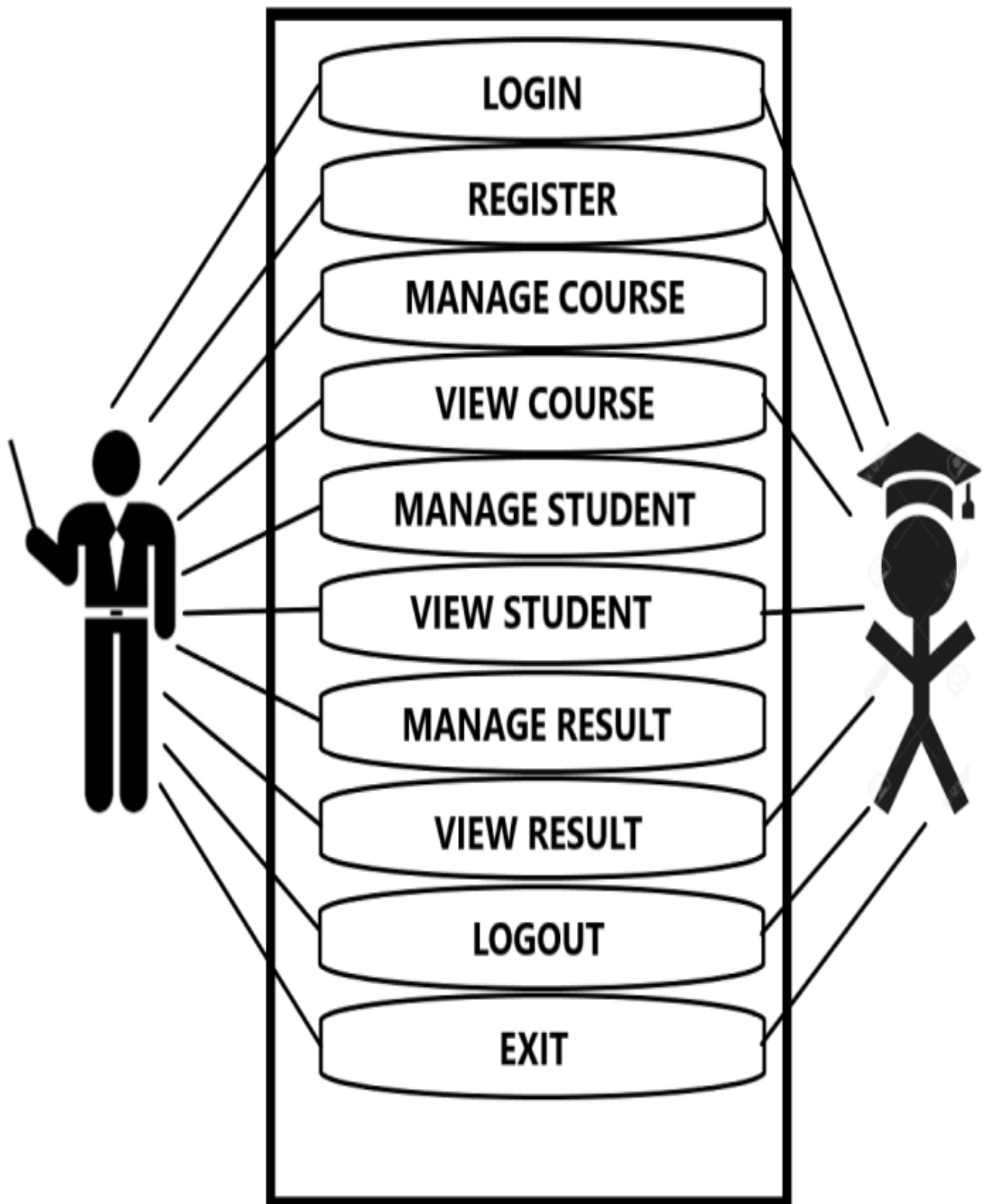
Processor: Intel P-IV System

Processor Speed: 250 MHz to 833 MHz

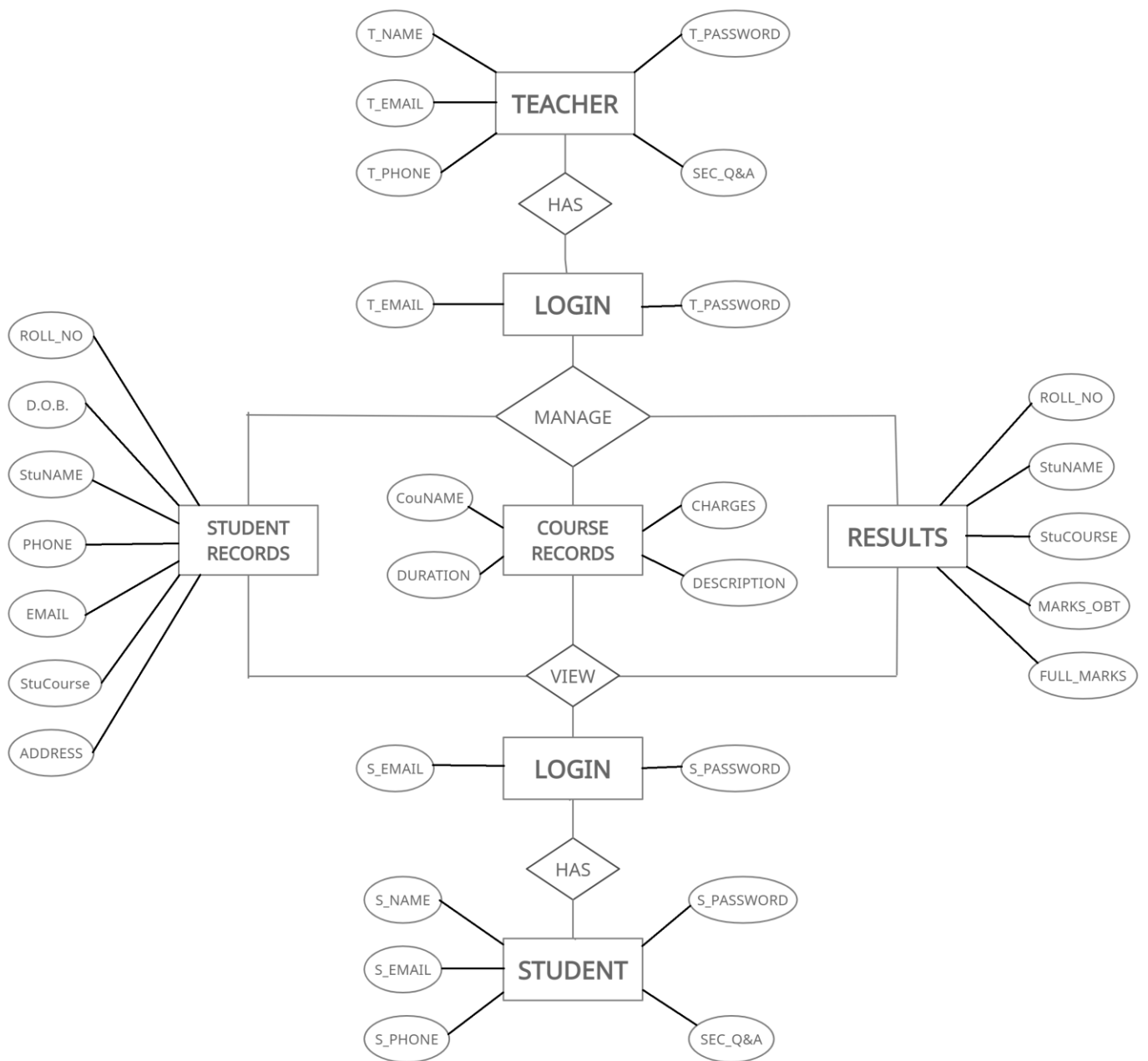
Ram: 512 Mb Ram

Hard Disk: 40 Gb

USE CASE DIAGRAM

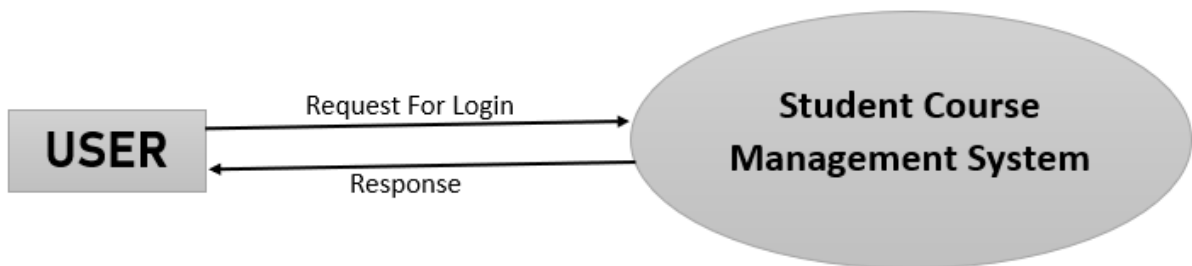


ER DIAGRAM

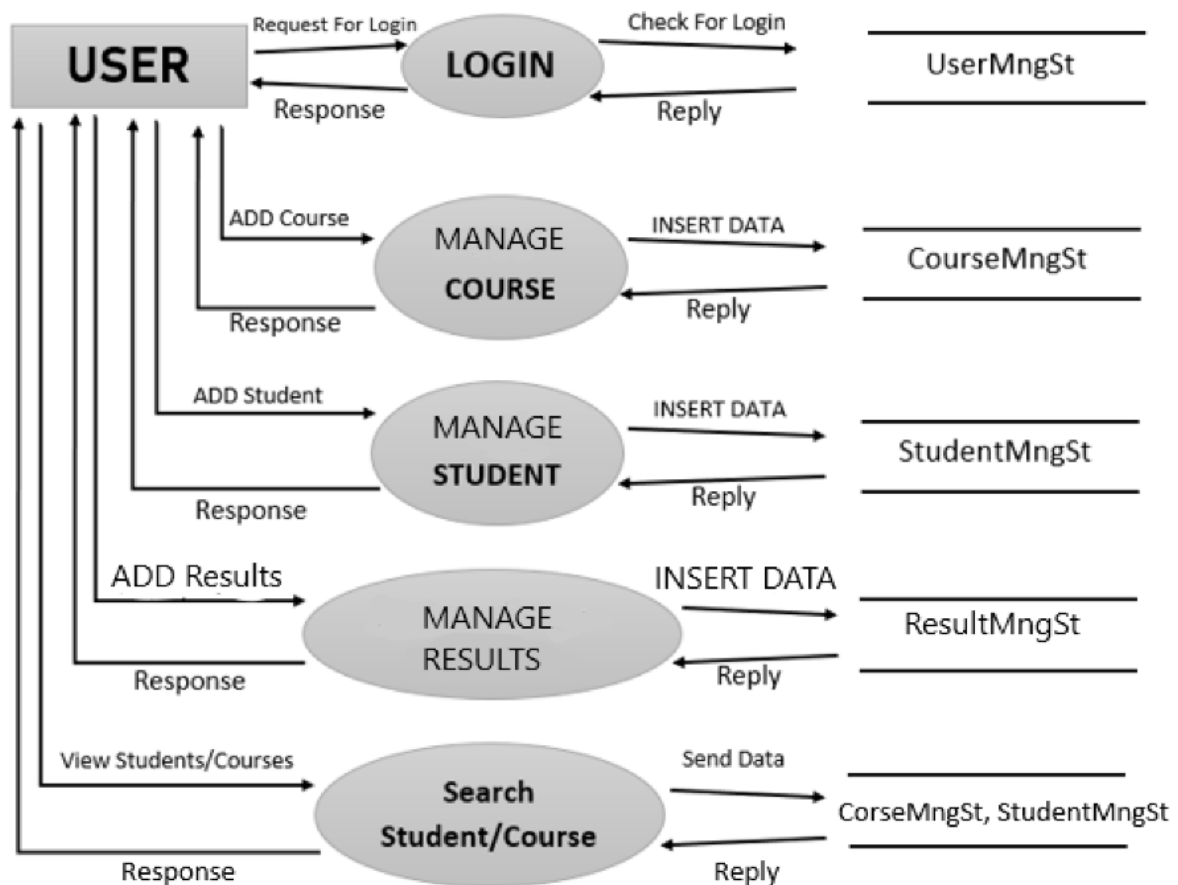


DATA FLOW DIAGRAM

LEVEL 0

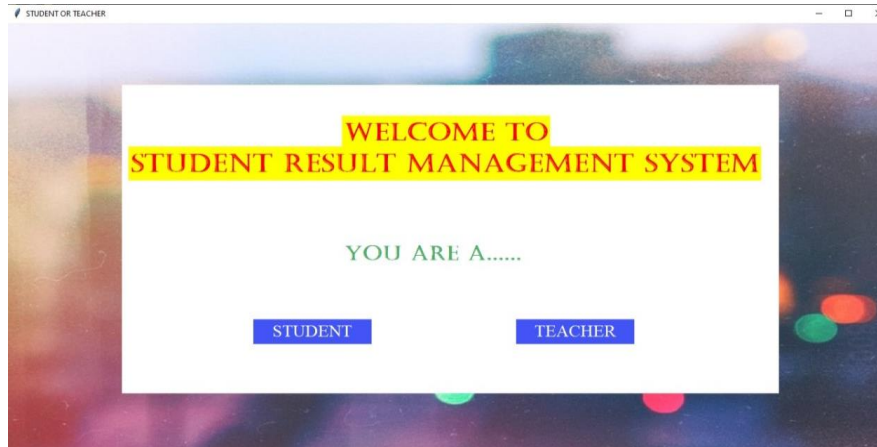


LEVEL 1

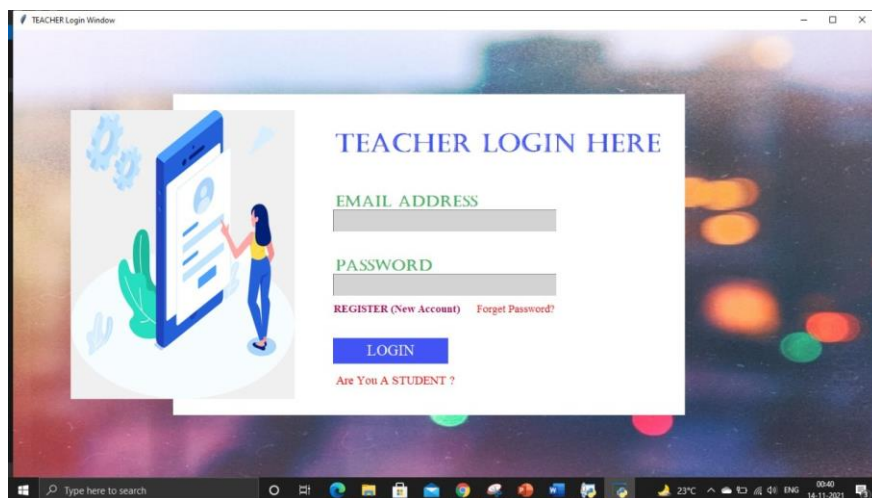


SCREENSHOTS

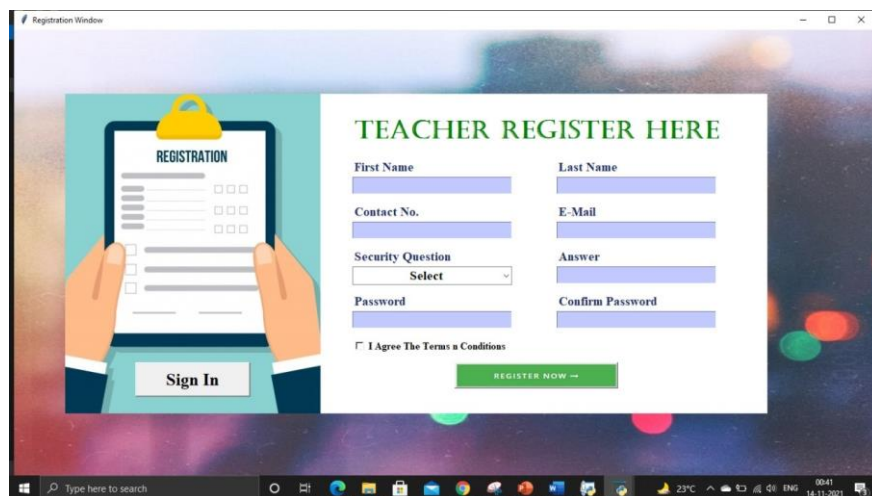
STUDENT OR TEACHER



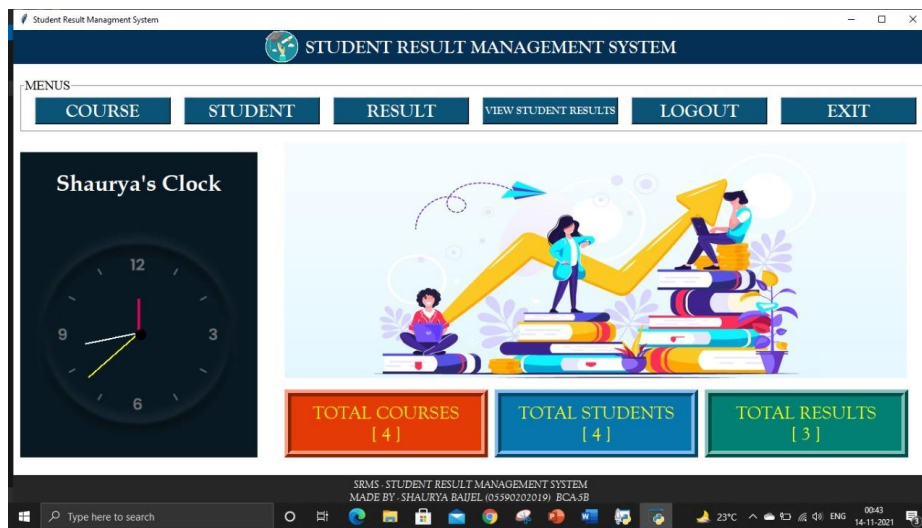
TEACHER/STUDENT LOGIN



TEACHER/STUDENT REGISTER



TEACHER DASHBOARD



MANAGE COURSE

The Manage Course Details form is shown within the "Student Result Management System" window. It includes a sidebar menu and a main form area. The form has input fields for Course Name (containing "R"), Duration (5 months), Charges (8000), and Description (xyz). A table on the right lists existing courses. At the bottom are buttons for SAVE, UPDATE, DELETE, and CLEAR. The footer text is: "SRMS - STUDENT RESULT MANAGEMENT SYSTEM MADE BY - SHAURYA BAIJEL (05590202019) BCA-5B".

Course Id	Name	Duration	Charges	Desc
1	Python	5 months	5000	aabbcc
2	Java	6 months	10000	abc
3	Sql	3 months	3000	axyzyzzz
4	R	5 months	8000	xyz

MANAGE STUDENTS

The Manage Student Details form is displayed within the "Student Result Management System" window. It features a sidebar menu and a main form area. The form contains input fields for Roll No (101), D.O.B. (02.08.2000), Name (Shaurya Baijel), Contact (7838430471), E Mail (sbaijel@gmail.com), Admission (02.08.2019), Gender (Male), Course (Python), State (Delhi), City (New Delhi), Pin (110007), and Address (ainiaiaiai). A table on the right lists existing students. At the bottom are buttons for SAVE, UPDATE, DELETE, and CLEAR. The footer text is: "SRMS - STUDENT RESULT MANAGEMENT SYSTEM MADE BY - SHAURYA BAIJEL (05590202019) BCA-5B".

Roll	Name	Email	Gender	D.O.B
101	Shaurya Baijel	sbaijel@gmail.com	Male	02.08.2000
102	Harsh Koli	hkolli@gmail.com	Male	16.11.20
103	Tarveen Kaur	tkaur@gmail.com	Female	01.01.20
104	Rohit Singh	rsingh@gmail.com	Others	06.06.20

MANAGE RESULTS

Student Result Management System

STUDENT RESULT MANAGEMENT SYSTEM

MENUS

COURSE **STUDENT** **RESULT** **VIEW STUDENT RESULTS** **LOGOUT** **EXIT**

Student Result Management System

ADD STUDENT RESULT

Select Student: 101 **SEARCH**

Name: Shaurya Baijel

Course: Python

Marks Obtained: 0

Full Marks: 10

SUBMIT **CLEAR**

SRMS - STUDENT RESULT MANAGEMENT SYSTEM
MADE BY: SHAURYA BAIJEL (05590202019) BCA-3B

Type here to search

23°C 00:46 14-11-2021

STUDENT DASHBOARD

Student Result Management System

STUDENT RESULT MANAGEMENT SYSTEM

MENUS

VIEW COURSES **VIEW STUDENTS** **VIEW RESULTS** **LOGOUT** **EXIT**

Shaurya's Clock



TOTAL COURSES [4]

TOTAL STUDENTS [4]

TOTAL RESULTS [3]

SRMS - STUDENT RESULT MANAGEMENT SYSTEM
MADE BY: SHAURYA BAIJEL (05590202019) BCA-3B

Type here to search

23°C 00:54 14-11-2021

VIEW COURSE

Student Result Management System

STUDENT RESULT MANAGEMENT SYSTEM

MENUS

VIEW COURSES **VIEW STUDENTS** **VIEW RESULTS** **LOGOUT** **EXIT**

Student Result Management System

Manage COURSE Details

Course Name: Python

Duration: 5 months

Charges: 5000

Description: aabbcc

CLEAR

Course Name: **SEARCH**

Course Id	Name	Duration	Charges	Descr
1	Python	5 months	5000	aabbcc
2	Java	6 months	10000	abc
3	Sql	3 months	3000	xxxxxyzzz
4	R	5 months	8000	xyz

SRMS - STUDENT RESULT MANAGEMENT SYSTEM
MADE BY: SHAURYA BAIJEL (05590202019) BCA-3B

Type here to search

23°C 00:55 14-11-2021

VIEW STUDENTS

Student Result Management System

STUDENT RESULT MANAGEMENT SYSTEM

MENUS

VIEW COURSES VIEW STUDENTS VIEW RESULTS LOGOUT EXIT

Student Result Management System

Manage STUDENT Details

Roll No: 101 D.O.B: 02.08.2000 Roll Number: SEARCH

Name: Shaurya Baijel Contact: 7838430471

E Mail: sbaijel@gmail.com Admission: 02.08.2019

Gender: Male Course: Python

State: Delhi City: New Delhi Pin: 110007

Address: aiaiaiaia

CLEAR

Roll	Name	Email	Gender	Di
101	Shaurya Baijel	sbaijel@gmail.com	Male	02.08.20
102	Hansh Koli	haskoli@gmail.com	Male	16.11.20
103	Tanveer Kaur	tkaur@gmail.com	Female	01.01.20
104	Rohit Singh	rsingh@gmail.com	Others	06.06.20

SRMS - STUDENT RESULT MANAGEMENT SYSTEM
MADE BY - SHAURYA BAIJEL (05590202019) BCA-5B

VIEW RESULTS

Student Result Management System

STUDENT RESULT MANAGEMENT SYSTEM

MENUS

VIEW COURSES VIEW STUDENTS VIEW RESULTS LOGOUT EXIT

Student Result Management System

VIEW STUDENT RESULT

SEARCH by Roll No: 101 SEARCH CLEAR

Roll No.	Name	Course	Marks Obtained	Total Marks	Percentage(%)
101	Shaurya Baijel	Python	10	10	100.0

SRMS - STUDENT RESULT MANAGEMENT SYSTEM
MADE BY - SHAURYA BAIJEL (05590202019) BCA-5B

CODINGS

LOGIN WINDOW

```
import os
from tkinter import *
from PIL import Image, ImageTk
from tkinter import ttk, messagebox
import sqlite3

class login_window:
    def __init__(self, root):
        self.root = root
        self.root.title("Login Window")
        self.root.geometry("1350x700+0+0")
        self.root.config(bg="white")

        #=====Background=====
        self.bg_img=Image.open("images/b2.jpg")
        self.bg_img=self.bg_img.resize((1350,700),Image.ANTIALIAS)
        self.bg_img=ImageTk.PhotoImage(self.bg_img)
        bg = Label(self.root, image=self.bg_img).place(x=0, y=0, relwidth=1, relheight=1)

        # =====Frames=====
        login_frame = Frame(self.root, bg="white")
        login_frame.place(x=250, y=100, width=800, height=500)

        title = Label(login_frame, text="LOGIN HERE", font=("Castellar", 30, "bold"), bg="white", fg="#4254f5").place(x=250,
y=50)

        email=Label(login_frame, text="EMAIL ADDRESS", font=("Castellar", 18, "bold"), bg="white",
fg="#53ad6b").place(x=250, y=150)
        self.txt_email=Entry(login_frame,font=("times new roman", 15, "bold"), bg="lightgray")
        self.txt_email.place(x=250,y=180,width=350,height=35)

        password=Label(login_frame, text="PASSWORD", font=("Castellar", 18, "bold"), bg="white",
fg="#53ad6b").place(x=250, y=250)
        self.txt_password=Entry(login_frame,font=("times new roman", 15, "bold"), bg="lightgray")
        self.txt_password.place(x=250,y=280,width=350,height=35)

        btn_reg=Button(login_frame,text="REGISTER (New Account)",command=self.regester_window,font=("times new
roman", 14),bg="white",bd=0,fg="red",cursor="hand2").place(x=250,y=320)
        btn_login=Button(login_frame,text="LOGIN",command=self.login,font=("times new roman",
18),bg="#4254f5",bd=0,fg="white",cursor="hand2").place(x=250,y=380,width=180,height=40)

        #=====login image=====
        self.left = Image.open("images/side2.png")
        self.left = self.left.resize((350, 450), Image.ANTIALIAS)
        self.left = ImageTk.PhotoImage(self.left)
        left = Label(self.root, image=self.left).place(x=90, y=120, width=350, height=450)

    def regester_window(self):
        self.root.destroy()
        os.system("python register.py")

    def login(self):
        if self.txt_email.get()==" or self.txt_password.get()=="":
```

```

        messagebox.showerror("Error", "All Fields Are Required", parent=self.root)
    else:
        try:
            con = sqlite3.connect(database="rms.db")
            cur = con.cursor()
            cur.execute("select * from employee where email=? and
password=?", (self.txt_email.get(), self.txt_password.get()))
            row = cur.fetchone()
            if row == None:
                messagebox.showerror("Error", "Invalid Username Or Password", parent=self.root)
            else:
                messagebox.showinfo("Success", "WELCOME", parent=self.root)
                self.root.destroy()
                os.system("python dashboard.py")
            con.close()
        except Exception as es:
            messagebox.showerror("Error", f"Error Due To: {str(es)}", parent=self.root)

root = Tk()
obj = login_window(root)
root.mainloop()

```

REGISTER WINDOW

```

from tkinter import *
from PIL import Image, ImageTk
from tkinter import ttk, messagebox
import sqlite3
import os

class Register:
    def __init__(self, root):
        self.root = root
        self.root.title("Registration Window")
        self.root.geometry("1350x700+0+0")
        self.root.config(bg="white")
        #=====BG Image=====

        self.bg_img = Image.open("images/b2.jpg")
        self.bg_img = self.bg_img.resize((1350, 700), Image.ANTIALIAS)
        self.bg_img = ImageTk.PhotoImage(self.bg_img)
        bg = Label(self.root, image=self.bg_img).place(x=0, y=0, relwidth=1, relheight=1)

        self.left = ImageTk.PhotoImage(file="images/side.png")
        left = Label(self.root, image=self.left).place(x=80, y=100, width=400, height=500)

        #=====Register Frame=====
        frame1 = Frame(self.root, bg="white")
        frame1.place(x=480, y=100, width=700, height=500)
        title = Label(frame1, text="REGISTER HERE", font=("Castellar", 30, "bold"), bg="white", fg="green").place(x=50, y=30)

        #row1=====
        f_name = Label(frame1, text="First Name", font=("times new
roman", 15, "bold"), bg="white", fg="#1e2c63").place(x=50, y=100)
        self.txt_fname = Entry(frame1, font=("times new roman", 15, "bold"), bg="#c2c9ff")
        self.txt_fname.place(x=50, y=130, width=250)

```

```

l_name=Label(frame1,text="Last Name",font=("times new
roman",15,"bold"),bg="white",fg="#1e2c63").place(x=370,y=100)
self.txt_lname=Entry(frame1,font=("times new roman",15,"bold"),bg="#c2c9ff")
self.txt_lname.place(x=370,y=130,width=250)

#row2=====
contact=Label(frame1,text="Contact No.",font=("times new
roman",15,"bold"),bg="white",fg="#1e2c63").place(x=50,y=170)
self.txt_contact=Entry(frame1,font=("times new roman",15,"bold"),bg="#c2c9ff")
self.txt_contact.place(x=50,y=200,width=250)

email=Label(frame1,text="E-Mail",font=("times new roman",15,"bold"),bg="white",fg="#1e2c63").place(x=370,y=170)
self.txt_email=Entry(frame1,font=("times new roman",15,"bold"),bg="#c2c9ff")
self.txt_email.place(x=370,y=200,width=250)

#row3=====
question=Label(frame1,text="Security Question",font=("times new
roman",15,"bold"),bg="white",fg="#1e2c63").place(x=50,y=240)

self.cmb_quest=ttk.Combobox(frame1,font=("times new roman",15,"bold"),state="readonly",justify=CENTER)
self.cmb_quest['values']=("Select","Your 1st Pet Name","Your Birth Place","Your Best Friend Name")
self.cmb_quest.place(x=50, y=270, width=250)
self.cmb_quest.current(0)

answer=Label(frame1,text="Answer",font=("times new
roman",15,"bold"),bg="white",fg="#1e2c63").place(x=370,y=240)
self.txt_answer=Entry(frame1,font=("times new roman",15,"bold"),bg="#c2c9ff")
self.txt_answer.place(x=370,y=270,width=250)

#row4=====
password=Label(frame1,text="Password",font=("times new
roman",15,"bold"),bg="white",fg="#1e2c63").place(x=50,y=310)
self.txt_password=Entry(frame1,font=("times new roman",15,"bold"),bg="#c2c9ff")
self.txt_password.place(x=50,y=340,width=250)

cpassword=Label(frame1,text="Confirm Password",font=("times new
roman",15,"bold"),bg="white",fg="#1e2c63").place(x=370,y=310)
self.txt_cpassword=Entry(frame1,font=("times new roman",15,"bold"),bg="#c2c9ff")
self.txt_cpassword.place(x=370,y=340,width=250)

#=====TERMS=====
self.var_chk=IntVar()
chk=Checkbutton(frame1,text="I Agree The Terms n
Conditions",variable=self.var_chk,onvalue=1,offvalue=0,bg="white",font=("times new
roman",12,"bold")).place(x=50,y=380)

self.btn_img=ImageTk.PhotoImage(file="images/register.png")

btn_register=Button(frame1,image=self.btn_img,bd=2,cursor="hand2",command=self.register_data).place(x=210,y=420)
btn_login=Button(self.root,text="Sign In",font=("times new
roman",20,"bold"),bd=2,cursor="hand2",command=self.login_window,relief=RAISED).place(x=190,y=520,width=180)

def clear(self):
    self.txt_fname.delete(0,END)
    self.txt_lname.delete(0,END)
    self.txt_contact.delete(0,END)
    self.txt_email.delete(0,END)
    self.txt_answer.delete(0,END)

```

```

self.txt_password.delete(0,END)
self.txt_cpassword.delete(0,END)
self.cmb_quest.current(0)

def register_data(self):
    if self.txt_fname.get()==" or self.txt_contact.get()==" or self.txt_email.get()==" or self.cmb_quest.get()=="Select" or
self.txt_answer.get()==" or self.txt_password.get()==" or self.txt_cpassword.get()=="":
        messagebox.showerror("Error", "All Fields Are Required",parent=self.root)

    elif self.txt_password.get()!=self.txt_cpassword.get():
        messagebox.showerror("Error", "Password N Confirm Password Should Be Same",parent=self.root)

    elif self.var_chk.get()==0:
        messagebox.showerror("Error", "Please Agree Our T&Cs",parent=self.root)

    else:
        try:
            con=sqlite3.connect(database="rms.db")
            cur = con.cursor()
            cur.execute("select * from employee where email=?", (self.txt_email.get(),))
            row=cur.fetchone()
            #print(row)
            if row!=None:
                messagebox.showerror("Error", "User Already Exist, Please Try With Another Email Id",parent=self.root)
            else:
                cur.execute("insert into employee (f_name,l_name,contact,email,question,answer,password)
values(?,?,?,?,?,?,?,?)",
                    (self.txt_fname.get(),
                     self.txt_lname.get(),
                     self.txt_contact.get(),
                     self.txt_email.get(),
                     self.cmb_quest.get(),
                     self.txt_answer.get(),
                     self.txt_password.get()
                    ))
                con.commit()
                con.close()
                messagebox.showinfo("Error", "Register Suceeeful",parent=self.root)
                self.clear()
                self.login_window()
        except Exception as es:
            messagebox.showerror("Error", f"Error Due To: {str(es)}", parent=self.root)

def login_window(self):
    self.root.destroy()
    os.system("python login.py")

root=Tk()
obj=Register(root)
root.mainloop()

```

DASHBOARD

```

import os
from tkinter import*
from PIL import Image,ImageTk,ImageDraw
from datetime import *
from time import *

```

```

from math import *
import sqlite3
from course import CourseClass
from student import StudentClass
from tkinter import messagebox,ttk

class RMS:
    def __init__(self,root):
        self.root=root
        self.root.title("Student Course Managment System")
        self.root.geometry("1350x700+0+0")
        self.root.config(bg="white")

        #====ICONS====
        self.logo_dash=ImageTk.PhotoImage(file="images/logo_p.png")

        #====TITLE====
        title=Label(self.root,text="STUDENT COURSE MANAGEMENT
SYSTEM",padx=10,compound=LEFT,image=self.logo_dash,font=("goudy old
style",20,"bold"),bg="#033054",fg="white").place(x=0,y=0,relwidth=1,height=50)

        #====MENU====
        M_Frame=LabelFrame(self.root,text=("MENUS"),font=("times new roman",15),bg="white")
        M_Frame.place(x=10,y=70,width=1340,height=80)

        btn_course=Button(M_Frame,text="MANAGE COURSE",font=("goudy old
style",20,"bold"),bg="#0b5377",fg="white",cursor="hand2",command=self.add_course).place(x=20,y=5,width=400,height=
40)
        btn_student=Button(M_Frame,text="MANAGE STUDENT",font=("goudy old
style",20,"bold"),bg="#0b5377",fg="white",cursor="hand2",command=self.add_student).place(x=460,y=5,width=400,height=
40)
        btn_logout=Button(M_Frame,text="LOGOUT",font=("goudy old
style",20,"bold"),bg="#0b5377",fg="white",cursor="hand2",command=self.logout).place(x=900,y=5,width=200,height=40)
        btn_exit=Button(M_Frame,text="EXIT",font=("goudy old
style",20,"bold"),bg="#0b5377",fg="white",cursor="hand2",command=self.exit_).place(x=1120,y=5,width=200,height=40)

        #====Content Window====
        self.bg_img=Image.open("images/bg.png")
        self.bg_img=self.bg_img.resize((920,350),Image.ANTIALIAS)
        self.bg_img=ImageTk.PhotoImage(self.bg_img)
        self.lbl_bg=Label(self.root,image=self.bg_img).place(x=200,y=165,width=920,height=350)

        #====UPDATE DATAILS====
        self.lbl_course=Label(self.root,text="TOTAL COURSES\n[ 0 ]",font=("goudy old
style",20),bd=10,relief=RIDGE,bg="#e43b06",fg="yellow")
        self.lbl_course.place(x=20,y=530,width=300,height=100)
        self.lbl_student=Label(self.root,text="TOTAL STUDENTS\n[ 0 ]",font=("goudy old
style",20),bd=10,relief=RIDGE,bg="#0676ad",fg="yellow")
        self.lbl_student.place(x=1020,y=530,width=300,height=100)

        #====FOOTER====
        footer=Label(self.root,text="SRMS - STUDENT COURSE MANAGEMENT SYSTEM\nMADE BY - SHAURYA BAIJEL
(05590202019) BCA-5B ",font=("goudy old style",12,"italic"),bg="#262626",fg="white").pack(side=BOTTOM,fill=X)
        self.update_details()

    def add_course(self):
        self.new_win=Toplevel(self.root)
        self.new_obj=CourseClass(self.new_win)

```



```

def add_student(self):
    self.new_win=Toplevel(self.root)
    self.new_obj=StudentClass(self.new_win)

def logout(self):
    op=messagebox.askyesno("Confirm","Do You Really Want To Logout ?",parent=self.root)
    if op==True:
        self.root.destroy()
        os.system("python login.py")

def exit_(self):
    op = messagebox.askyesno("Confirm", "Do You Really Want To EXIT ?", parent=self.root)
    if op == True:
        self.root.destroy()

def update_details(self):
    con = sqlite3.connect(database="rms.db")
    cur = con.cursor()
    try:
        cur.execute("select * from course")
        cr = cur.fetchall()
        self.lbl_course.config(text=f"TOTAL COURSES\n[ {str(len(cr))} ]")
        cur.execute("select * from student")
        cr = cur.fetchall()
        self.lbl_student.config(text=f"TOTAL STUDENTS\n[ {str(len(cr))} ]")
        self.lbl_course.after(200,self.update_details)

    except Exception as ex:
        messagebox.showerror("Error", f"Error due to {str(ex)}")

#====CONNECTION CLOSE====
if __name__=="__main__":
    root=Tk()
    obj=RMS(root)
    root.mainloop()

```

CREATE DATABASES

```

import sqlite3

def create_db():
    con=sqlite3.connect(database="rms.db")
    cur=con.cursor()
    cur.execute("CREATE TABLE IF NOT EXISTS course(cid INTEGER PRIMARY KEY AUTOINCREMENT,name text,duration
text,charges text,description text)")
    con.commit()

    cur.execute("CREATE TABLE IF NOT EXISTS student(roll INTEGER PRIMARY KEY AUTOINCREMENT,name text,email
text,gender text,dob text,contact text,admission text,course text,state text,city text,pin text,address text)")
    con.commit()

    cur.execute("CREATE TABLE IF NOT EXISTS employee(eid INTEGER PRIMARY KEY AUTOINCREMENT,f_name text,l_name
text,contact text, email text, question text, answer text, password text)")
    con.commit()
    con.close()

create_db()

```

CONCLUSION

Student Management System can be used by educational institutions to maintain their student records easily. Achieving this objective is difficult using the manual system as the information is scattered, can be redundant, and collecting relevant information may be very time-consuming. All these problems are solved by this project.

The system is developed to cope up with the current issues and problems of managing Student Records. The system can add, delete, update Student Records and Courses Records.

After computerized system is implemented less human force will be required to maintain the Student Records, thus reducing the overall cost.

User is able to search student record by using few clicks of mouse and few search keywords thus saving their valuable time.

This system helps in maintaining the information of pupils of the organization. It can be easily accessed by the manager and kept safe for a long period of time without any changes.

FUTURE SCOPE

- ❖ In Future, we will add forget password option in which otp is sent on email id.
- ❖ In Future, we will add a printing option for printing all the records of the student and their respective courses.
- ❖ In Future, we will take the system online so that anyone on the internet can access our system to manage students for their institutes, schools, etc.
- ❖ In Future, we will add course study materials for the students who register in this system.